



AvenirVista
Tech Solutions

Feasibility Check for Project AEGIS

Document to Vendor – Information for Sample Application Development

Objective:

Establish a **simple end-to-end prototype application** to validate real-time image capture, backend inference integration, and low-latency visualization using a **publisher-subscriber communication model**.

The intent is **not productization**, but to **demonstrate technical feasibility**, latency compliance, and architectural soundness.

1. Front-End Application (Capture & Display)

- Develop a lightweight application with a **Start / Stop control**.
- On *Start*, the application shall:
 - Continuously capture images from the camera source.
 - Act as an **image publisher** to the backend.
- UI requirements:
 - Display **two image streams side-by-side**:
 - **Live Capture View** – current frame from the camera.
 - **Processed View** – previous frame super-imposed with detection results.
 - Ensure visual continuity and responsiveness suitable for real-time perception use cases.

2. Backend Integration (Provided by AvenirVista)

- A **sample Python backend script** is provided along with this document, containing:
 - YOLOv5-based object detection model.
 - A callable interface that:
 - Accepts **one image per request**.
 - Returns:
 - Bounding box coordinates.
 - Associated class labels.
- Vendor responsibility:
 - Integrate with the backend **as-is**, without modifying core model logic.
 - Treat the backend as a **subscriber to image data** and **publisher of inference results**.

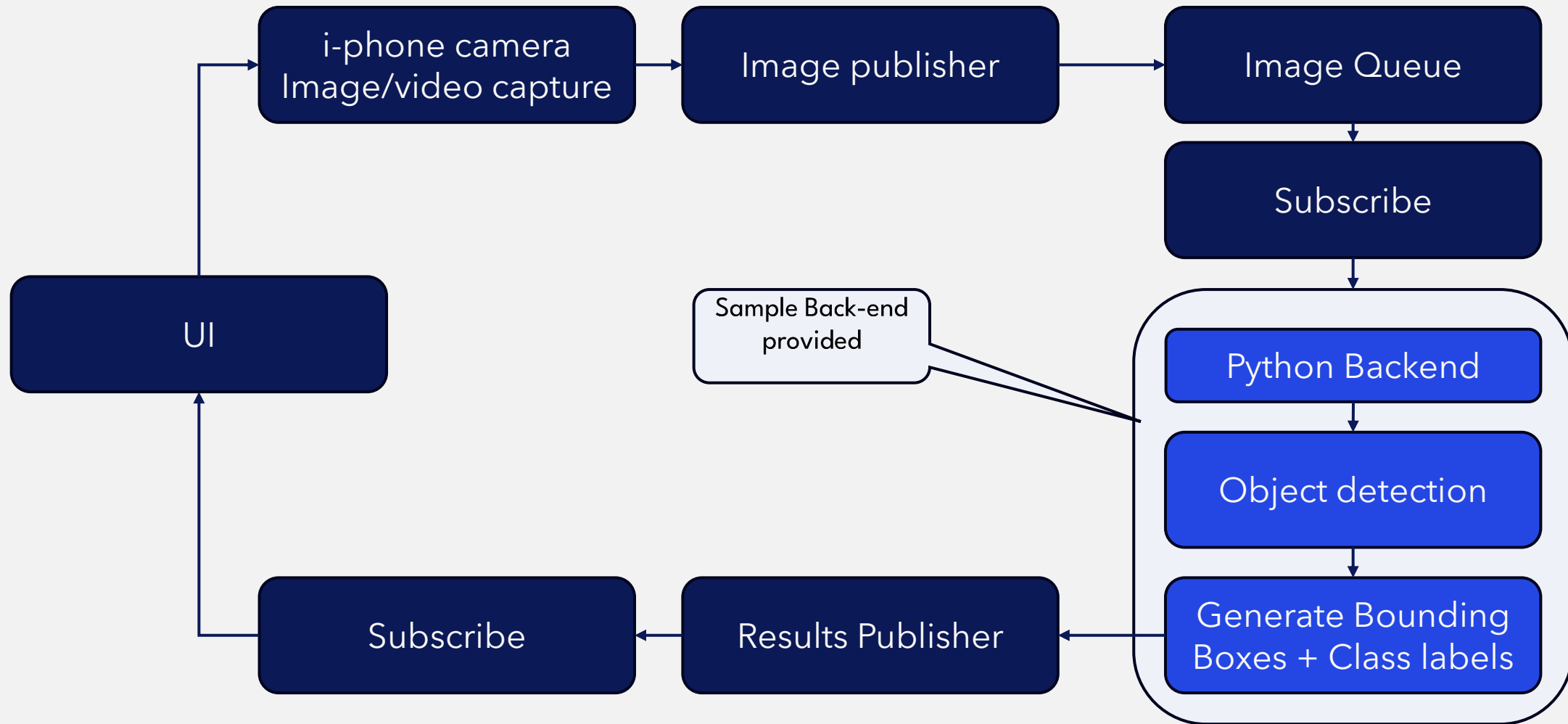
(Backend setup, usage, and invocation details are already outlined in the shared presentation)

3. Communication Architecture

- Implement a **publisher-subscriber protocol** between:
 - Front-end (Image Publisher / Result Subscriber)
 - Backend (Image Subscriber / Result Publisher)
- Technology choice is open (for protocol), provided:
 - The design supports **non-blocking operation**.
 - It is scalable enough for continuous streaming.
 - Note: Please evaluate if we can use protobuf
- Clear separation between:
 - Image transport
 - Inference execution
 - Result visualization

Scope of Work:

Information: Block Diagram



Scope of Work:

4. Post-Processing & Visualization

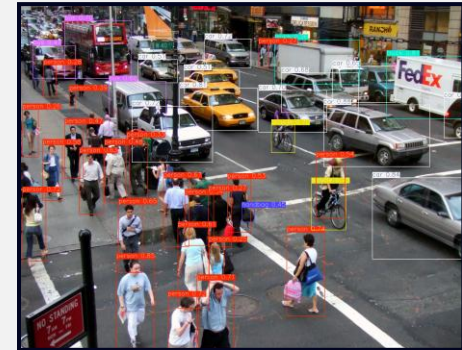
- On receiving backend results:
 - Superimpose bounding boxes and class labels on the corresponding image frame.
 - Render the annotated image in the **Processed View** panel.
- The processed image must correspond to the **immediately previous frame**, ensuring temporal consistency.

Image here is for sample purpose, we can capture indoor and check the results – model can detect 80 classes and provide output

Input Image



Processed Image



5. Performance & Latency Constraints

- End-to-end delay (capture → backend → visualization) shall be:
 - ≤ 100 ms, or
 - Bounded by the **actual model inference time**, whichever is higher.
- No frame blocking or UI freezing is acceptable during continuous operation.

6. Deliverables

- Working prototype demonstrating:
 - Continuous capture
 - Backend inference invocation
 - Publisher–subscriber communication
 - Side-by-side live and processed visualization
- Brief technical note covering:
 - Chosen communication mechanism
 - Observed latency (capture → inference → display)
 - Key limitations or improvement areas
- **Additionally, also please show us the log of other sensors, if possible, just to check the data dump for every 100 msecs**

Back-end Usage Information:

1. Setup

- **Install python 3.10.11(enable source path while installing)**
 - **Open folder(object_detection backend)**
- In terminal**
- **"python -m venv .venv"**
 - **" Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass"**
 - **" .\.venv\Scripts\Activate.ps1"**
 - **"pip install -r requirements.txt"**

2. Get Bounding Boxes

- **python main.py --weights weights/yolov5n.onnx --source image.jpg**

3. Get Image + Boxes

- **python main.py --weights weights/yolov5n.onnx --source image.jpg --draw**

Images Saved in runs/exp/ folder

Thank You!